

Authentication

Dal form di login alle API protette



Obiettivo della lezione

1. Form

Costruiamo una pagina con email e password.

2. POST /login

Il backend verifica e risponde con { token }.

3. Bearer

Il client usa
Authorization: Bearer
<token>.

Alla fine gli studenti devono capire il giro completo: login -> token -> fetch -> middleware.

Authentication vs Authorization

Authentication

Chi sei?

Esempio: email + password
dimostrano che sei
mario@example.com.

Authorization

Cosa puoi fare?

Esempio: puoi leggere /api/profile,
ma non i dati di altri utenti.

**Login = authentication. Accesso alle API =
authorization.**

HTTP e' stateless



Il server non ricorda automaticamente chi ha fatto login prima

Ogni richiesta deve portare una prova

La prova puo' essere una sessione/cookie o un token

Sessione, API key, token

Sessione + cookie

Il server crea una sessione.

Il browser manda il cookie automaticamente.

API key

Identifica un'app o progetto.

Non identifica bene un singolo utente.

Bearer token

Il server lo emette dopo il login.

Lo usiamo per proteggere le API.

Cookie di sessione



Nel browser

Il cookie viene salvato dal browser e reinviato automaticamente allo stesso dominio.

Sul server

Il server usa il valore del cookie per ritrovare la sessione associata all'utente.

Sessione: cookie -> utente

Cookie

sessionId=abc123

Sta nel browser.
Non contiene per forza i
dati utente.

Session store

abc123 -> userId 42

Mappa salvata sul
server o su Redis/DB.

req.user

Il middleware recupera
la sessione e sa chi ha
fatto la richiesta.

**Il cookie identifica la sessione; l'associazione sessione-utente
vive lato server.**

Sessioni in Express

```
app.use(session({
  secret: 'una stringa segreta',
  resave: false, saveUninitialized: false
}));
app.post('/login', (req, res) => {
  req.session.userId = user.id;
  res.redirect('/profile');
});
```

Il client vede solo il cookie. Il server ricorda cosa significa quel cookie.

Dove salvare lo stato di login?

Cookie

Inviato automaticamente dal browser.

Puo' essere HttpOnly e Secure.

Ottimo per sessioni web.

localStorage

Facile da usare con fetch.

JavaScript puo' leggerlo.

Comodo per demo, piu' esposto a XSS.

sessionStorage

Simile a localStorage, ma dura per la singola tab.

Sempre leggibile da JavaScript.

Cookie HttpOnly e Secure

```
Set-Cookie: sid=abc123; HttpOnly; Secure
```

```
Set-Cookie: sid=abc123; SameSite=Lax; Path=/
```

HttpOnly

JavaScript non può leggere il cookie: aiuta contro furti via XSS.

Secure

Il cookie viaggia solo su HTTPS.

SameSite

Riduce invii cross-site e aiuta contro CSRF.

API key

```
GET /api/meteo
Authorization: ApiKey 1234567890abcdef
x-api-key: 1234567890abcdef
```

Da ricordare

Una API key autentica l'applicazione, non il login del singolo utente.

Non va pubblicata nel codice.

Bearer token



```
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

Chi porta il token viene accettato

Per questo il token va protetto e deve scadere

Header

{ alg: 'HS256' }

Come e' firmato il token.

Payload

{ sub: 42, exp: ... }

Dati minimi sull'utente.

Signature

Firma crittografica.

Scopre se il token e' stato modificato.

Un JWT e' codificato, non cifrato: non metterci password o dati sensibili.

Bearer non significa JWT

Bearer

E' il modo in cui il token viene trasportato.

Header:

Authorization: Bearer <token>

Dice: chi porta il token puo' accedere.

JWT

E' un formato possibile del token.

Contiene header, payload e firma.

Il server puo' verificarlo senza cercare una sessione.

Bearer = schema HTTP. JWT = formato del token.

Quando usare sessione, Bearer, JWT

Sessione + cookie

App web tradizionale.

Il browser gestisce i cookie.

Il server mantiene lo stato.

Bearer token

API chiamate via fetch o da app/mobile.

Il client decide quando inviare l'header Authorization.

JWT

Buono quando vuoi token firmati e autodescrittivi.

Attenzione a scadenza e revoca.

Il flusso che implementiamo



Il login non apre magicamente tutte le API

Il token viene allegato alle richieste successive

Le route protette usano tutte lo stesso middleware

POST /login

```
app.post('/login', async (req, res) => {  
  const { email, password } = req.body;  
  const user = await findUserByEmail(email);  
  if (!user || !checkPassword(password, user))  
    return res.sendStatus(401);  
  const token = createToken({ sub: user.id });  
  res.json({ token });  
});
```

Client: chiamare una API protetta

```
const token = localStorage.getItem('token');  
const res = await fetch('/api/profile', {  
  headers: {  
    Authorization: `Bearer ${token}`  
  }  
});
```

Nota

Per una demo
localStorage e' comodo.

In produzione va
discusso con XSS,
cookie httpOnly e
HTTPS.

Middleware verifyToken

```
function verifyToken(req, res, next) {  
  const auth = req.headers.authorization;  
  if (!auth?.startsWith('Bearer '))  
    return res.sendStatus(401);  
  req.user = jwt.verify(auth.slice(7), SECRET);  
  next();  
}  
app.get('/api/profile', verifyToken, handler);
```

OAuth: forse dopo

Login nostro

Gestiamo utenti, password, token e middleware.

Buono per capire il meccanismo end to end.

OAuth

L'utente autorizza l'app tramite Google, GitHub, ecc.

Utile, ma richiede un flusso più lungo.

Prima capiamo bene i token. OAuth diventa molto meno misterioso dopo.

Mini app finale

Frontend

login.html

fetch('/login')

fetch('/api/profile')

Backend

POST /login

GET /api/profile

verifyToken

Concetti

password hash

token con scadenza

logout